AI Project Phase 2 Report

Group 5
Mani Abedii, Armin Fakhar, Kazem Motealehi

MODEL EXPLANATION

Model-1

The final model is a Feedforward Artificial Neural Network composed of:

- Input Layer
- Hidden Layer 1: 64 neurons, ReLU activation
- Hidden Layer 2: 32 neurons, ReLU activation
- Output layer with Sigmoid activation

Because of the label imbalance, we applied SMOTE Oversampling. This generates new samples of the minority class.

Before oversampling:

- Model predicts too many zeros
- Recall for churn becomes very low

After oversampling:

- Model becomes more sensitive to churn
- Recall increases

Model-2

The final model is a Feedforward Artificial Neural Network composed of:

- Input Layer
- 2 Hidden layers with 6 neurons and ReLU activation
- Output layer with Sigmoid activation

In Hidden layers ReLU activation was used because:

- It introduces non-linearity
- It prevents vanishing gradients
- It allows the model to learn complex patterns

Output Layer is a single neuron with **Sigmoid activation** outputs the probability that a customer will churn.

Because of the 80/20 label imbalance, we applied Class Weights in model-2. This penalizes misclassification of churn customers more heavily.

Without class weights:

- Model predicts too many zeros
- Recall for churn becomes very low

With class weights:

- Model becomes more sensitive to churn
- Recall increases
- Precision decreases (trade-off)

The model was trained using:

- Adam optimizer (learning rate = 0.001)
- Batch size = 32
- Maximum 100 epochs
- Early stopping to prevent overfitting
- Model checkpointing to save the best model

This ensures:

- Stable gradient updates
- Prevention of unnecessary
- Best validation performance is preserved

By default, classification uses threshold = 0.5. However, due to imbalance and business objectives, we evaluated multiple thresholds and selected the threshold that maximized F1-score on validation data. This allowed us to balance:

- Precision (avoiding FNs)
- Recall (detecting churn customers)

With these changes the final model does not aggressively over-predict churn and does not ignore churn customers. Just to mention that if we wanted higher recall we could lower the threshold but the precision would drop. Since the objective was maximizing F1-score (balanced performance), the current threshold is appropriate.

TRAINING EXPLANATION

Model-1 & Model-2

The model was trained to learn a mapping from customer features to a churn probability: **P(y = 1 | X)**
Because churn prediction is a binary classification task, the model outputs a single probability between 0 and 1, and training adjusts the network weights to make these probabilities match the true labels.

Loss Function:
We trained the model using **Binary Cross-Entropy (BCE)** loss:

$$\mathcal{L} = -\left(y \log(\hat{p}) + (1 - y) \log(1 - \hat{p})\right)$$

This loss is appropriate because:

- It penalizes incorrect probability estimates
- It encourages confident correct predictions

This is the standard and default loss for binary probability models.

As said before, Since the dataset is imbalanced (majority class 0), the model could achieve high accuracy by predicting “0” most of the time, which would hurt churn detection. To prevent this, we applied class weights during training. Errors on churn customers (class 1) were penalized more than errors on non-churn customers (class 0). This encourages the model to pay more attention to the minority class.

In practice, this changes the loss into a weighted version so churn mistakes become more “costly” and the model becomes more sensitive to churn patterns.

We used the **Adam optimizer** with a learning rate of 0.001.

Adam is an adaptive gradient-based method that:

- Updates weights efficiently
- Adjusts learning rates automatically per parameter
- Works well for noisy gradients and tabular neural networks

Each update step uses a mini-batch of data to estimate the gradient and improve the network weights. The batch-size=32, up to 100 epochs. This means:

- The model processes 32 examples at a time
- One epoch = one full pass over the training set
- Multiple epochs allow iterative refinement of the weights

A validation set was used during training to measure generalization performance. Validation performance is important because training metrics can improve even when the model is overfitting and validation metrics reflect unseen data behavior.

So after each epoch, the model computed:

- validation loss
- validation accuracy
- validation precision/recall
- validation AUC

This allowed us to detect overfitting and choose the best training endpoint.

ModelCheckpoint saved the best-performing model (based on validation loss) during training. This guarantees that we keep the best version of the model even if later epochs become worse. After training, the model outputs probabilities. The final step was converting probabilities into class labels.

Instead of using the default threshold 0.5, we tested many thresholds and selected the one that maximized the F1-score on validation data.

This helped us achieve the desired balance between:

- precision (reducing false positives)
- recall (reducing false negatives)

PLOTS & CHARTS

Model-1

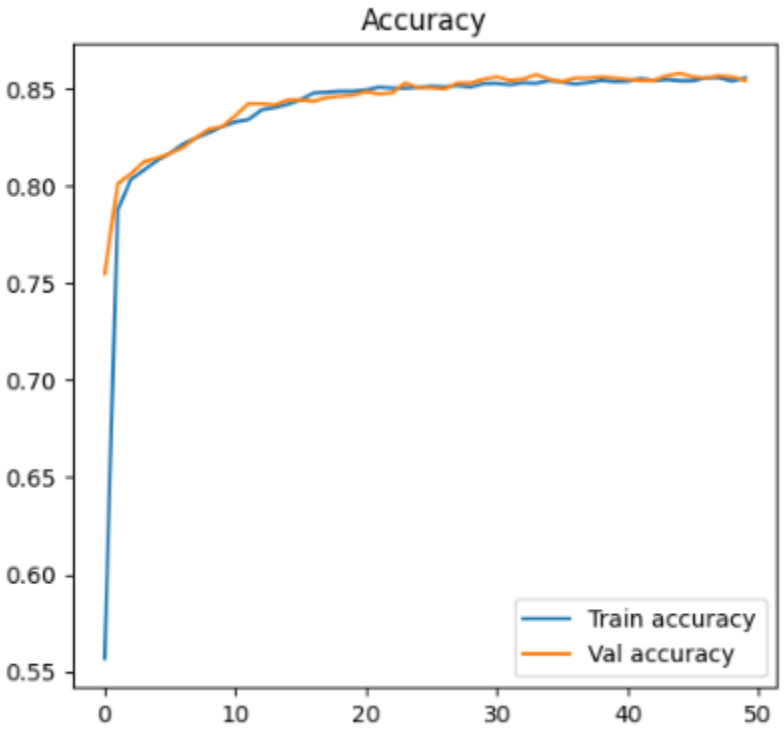


FIGURE (1.1): Validation Accuracy vs Train Accuracy

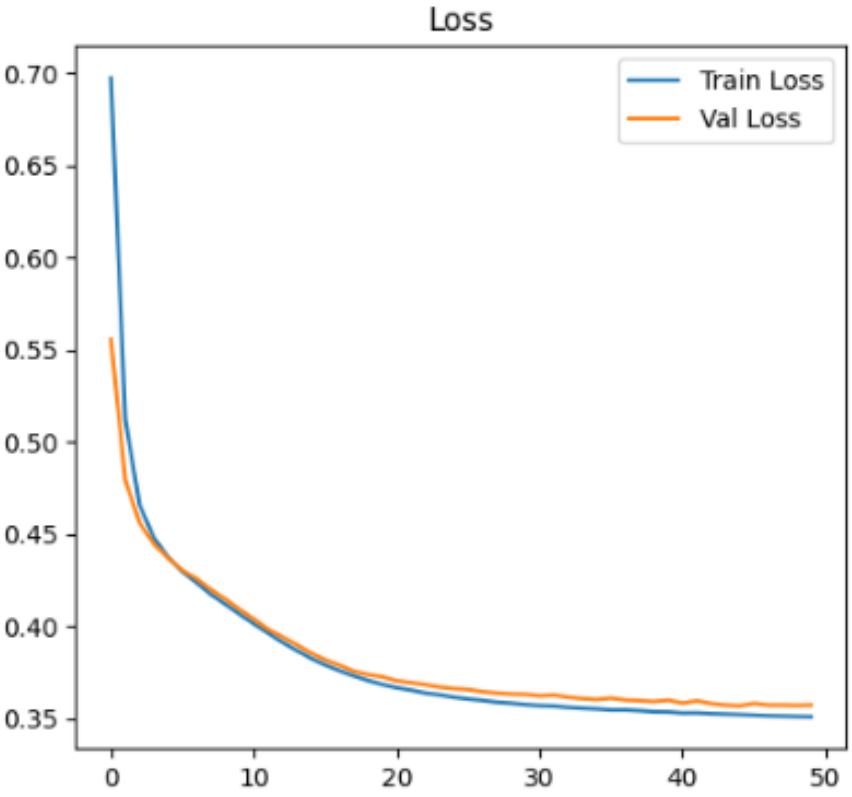


FIGURE (1.2): Train Loss vs. Validation Loss

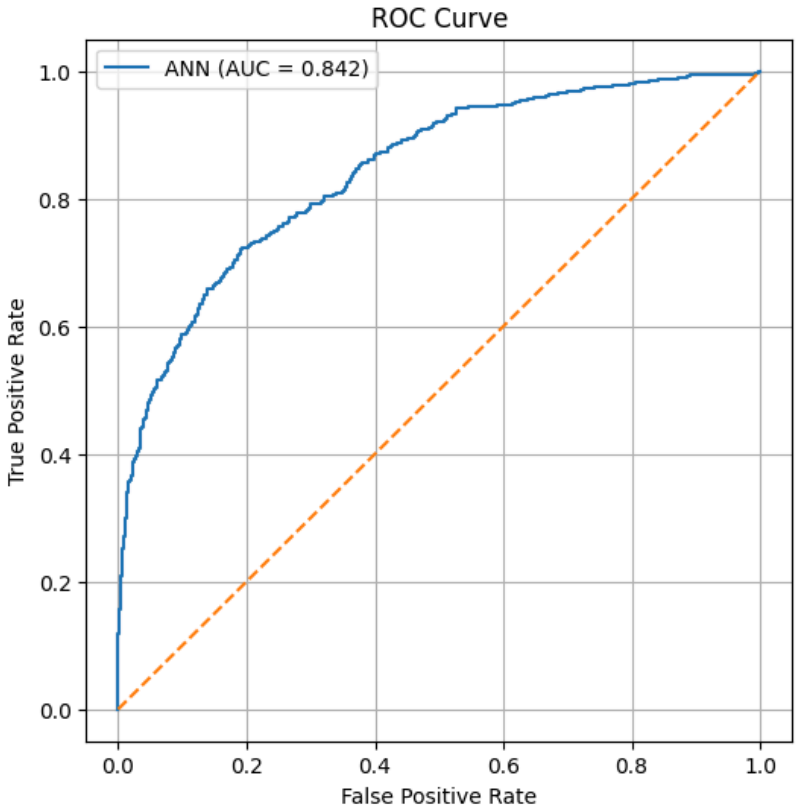


FIGURE (1.3): ROC Curve

Model-2

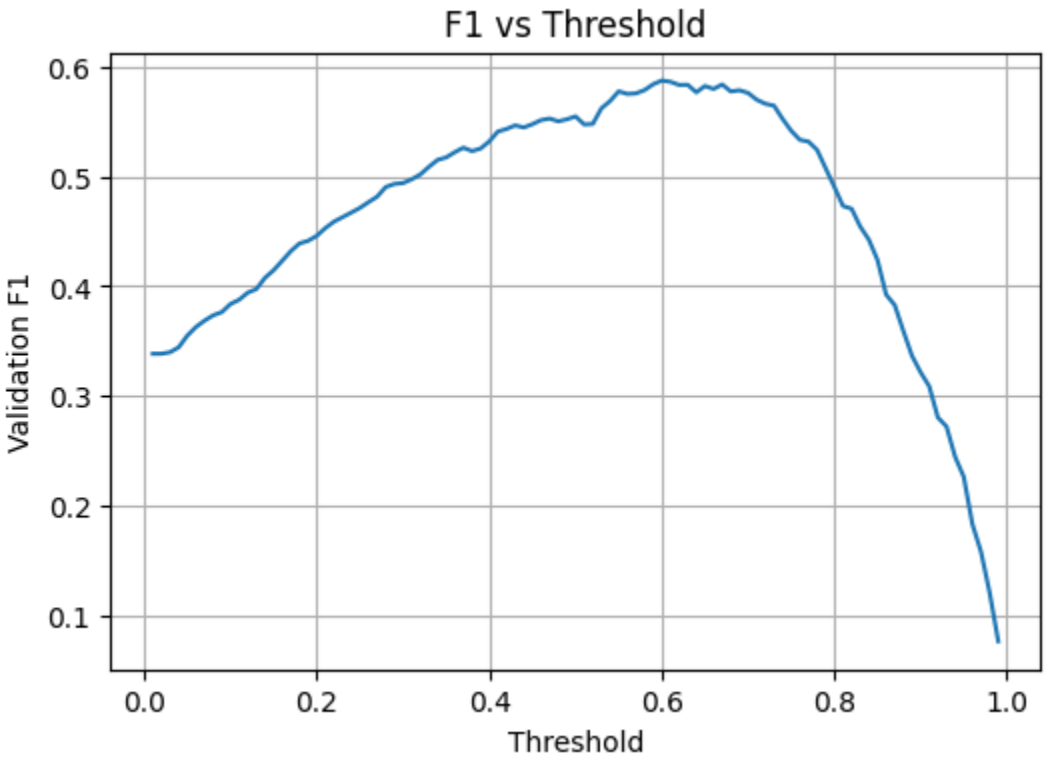


FIGURE (2.1): Validation F1-score vs. Threshold

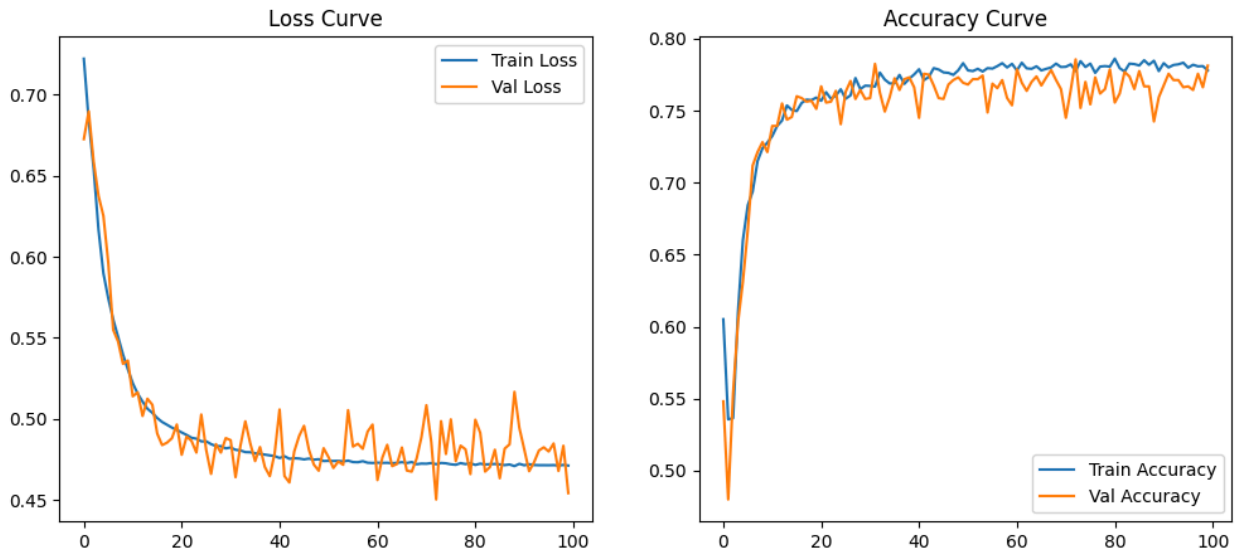


FIGURE (2.2): Validation & Train Loss Curve

FIGURE (2.3): Validation & Train Accuracy Curve

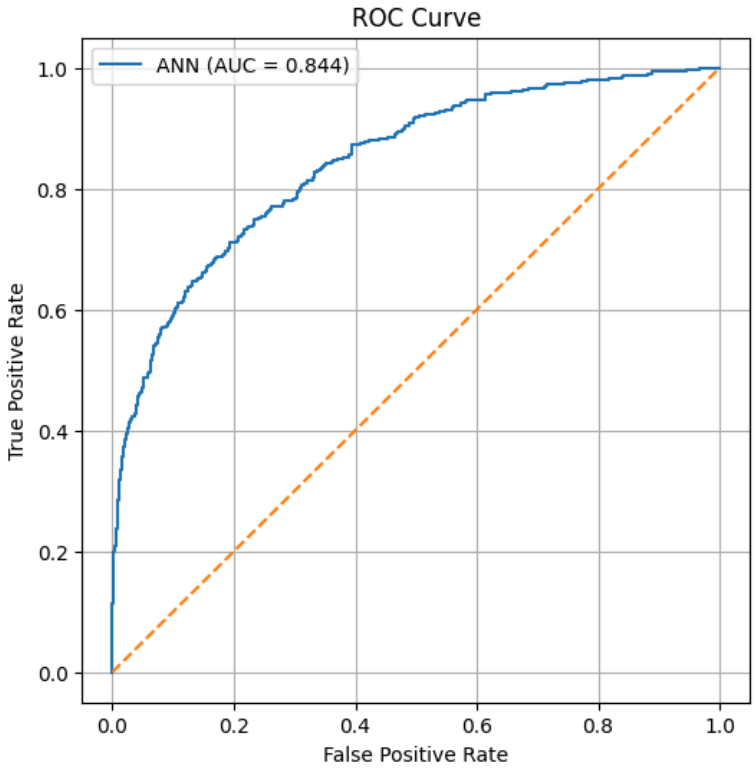


FIGURE (2.4): ROC Curve

FAULT ANALYSIS

Model-1 & Model-2

Every binary classification model makes two critical kind of mistakes:

False Positives (FP): Predicts churn (1) for a customer who actually stays (0).
Impact: unnecessary retention cost (discounts/calls/offers), wasted resources.

False Negatives (FN): Predicts stay (0) for a customer who actually churns (1). Impact: missed churn prevention opportunity (usually more costly).

Because the dataset is imbalanced, these error types do not carry the same practical risk. The model's threshold determines how it trades FP vs FN.

Confusion Matrix:

[[1418 175] ← Model-1

 [158 249]] → TN = 1414, FP = 175, FN = 158, TP = 249

[[1411 182] ← Model-2

 [155 252]] → TN = 1411, FP=182, FN=158, TP=252

False Positive Rate (for class 0)

Out of 1593 non-churn customers, 179 were predicted as churn →

FP rate = 175 / 1593 = 10.9% (model-1)

FP rate = 182 / 1593 = 11.4% (model-2)

→ The model incorrectly flags about 11% of stable customers as churn. This is relatively controlled and not excessive.

False Negative Rate (for class 1)

Out of 407 churn customers, about 158 were missed →

FN rate = 158 / 407 ≈ 38.8% (model-1)

FN rate = 155 / 407 ≈ 38.08% (model-2)

➜ The model misses nearly 38% of churners.

This is the main weakness of the model.

The dominant fault is “Missed churners”. Even though recall improved compared to earlier experiments, the model still fails to detect more than one-third of actual churn customers.

ROC-AUC about 0.84 indicates that the model separates churn vs non-churn reasonably well. There is predictive signal in features and the ranking ability is strong.

Using different thresholds in both models made the model more conservative. Compared to 0.5, fewer false positives & slightly fewer true positives were predicted and the precision is more controlled. This explains Precision = 0.58 and Recall = 0.61.

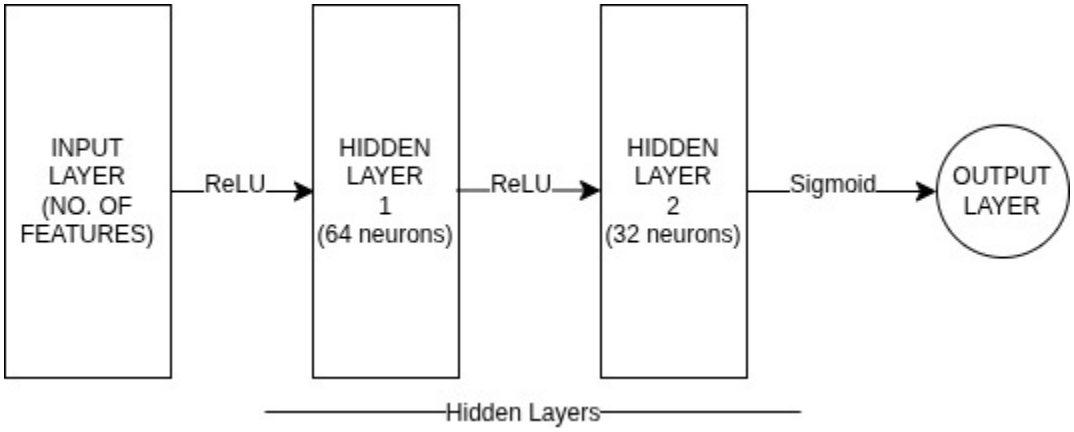
We achieved a **balanced point**, but at the cost of missing ~38% of churners. But, we are confident to say that this model no longer heavily favors one side.

→ As we discussed before, if the business cost of missing churners is high, 38% FN might still be too large but if the business cost of unnecessary retention action is high, 11% FP is acceptable and controlled.

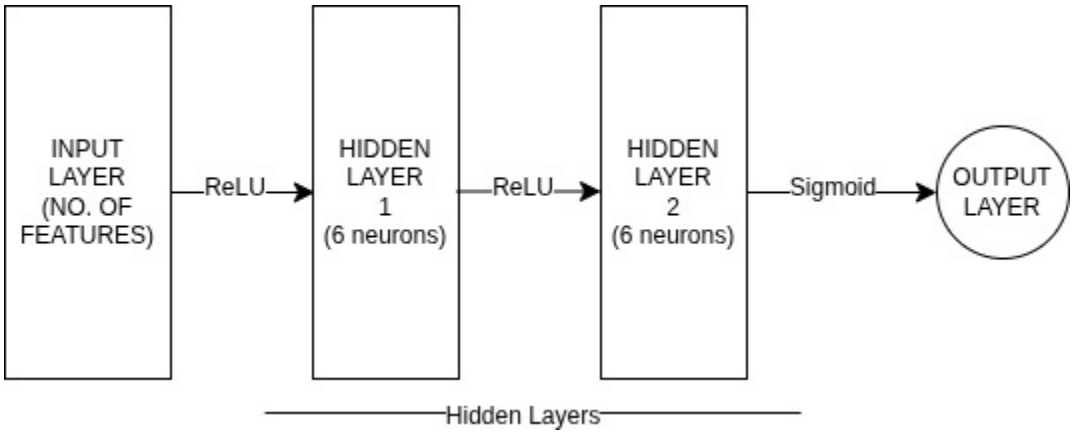
So whether this is “good” depends on business tolerance.

MODEL ARCHITECTURE

Model-1



Model-2



The final selected model is a compact Feedforward Artificial Neural Network with the following structure:

- **Input Layer** (dimension = number of processed features)
- **Hidden Layer 1:** 64 neurons in model-1, 6 neurons in model-2, ReLU activation
- **Hidden Layer 2:** 32 neurons in model-1, 6 neurons in model-2, ReLU activation
- **Output Layer:** 1 neuron, Sigmoid activation

Why a Small Architecture in Model-2?

During earlier experiments, we tested deeper and wider networks (e.g., 128-64-32). Although those models significantly reduced training loss, they showed clear overfitting behavior:

- Training loss decreased aggressively
- Validation loss increased
- Test F1-score did not improve

This indicates that **model capacity was not the limiting factor**.

Therefore, we reverted to a smaller architecture because:

- It generalizes similarly to the larger network
- It reduces overfitting risk
- It is computationally efficient
- It is easier to interpret and deploy

This follows an important ML principle:

> Increasing model complexity does not guarantee better generalization.

According to the fact that although model-1 is more complex compared to model-2, results don't have noticeable differences.

MODELS COMPARISON

We actually built two conceptually different systems:

- **Model 1** → Larger network (64-32), trained with SMOTE
- **Model 2** → Small network (6-6), trained with class weights

This is an important training difference.

Model-1 this architecture has larger capacity, more parameters, more expressive and uses SMOTE. Model–2 is a very small network with low parameter count. So the biggest conceptual difference is how they handle imbalance.

Model-1,

- Creates synthetic churn samples
- Changes the data distribution
- The model sees more churn-like patters
- Learns decision boundary from resampled data

So the model sees more minority examples and often improves recall. But, synthetic samples may not reflect real distribution and can cause distort probability calibration.

Model-2,

- Does not change data
- Modifies the loss function
- Penalizes churn misclassification more heavily

So this model preserves real data distribution, is a more cleaner theoretical approach and is a better probability calibration. But, it can create unstable precision-recall trade-offs and does not increase actual minority diversity.

Both models ended up with:

- ROC-AUC = 0.84
- Macro F1 = 0.75
- Churn F1 = 0.59-0.61

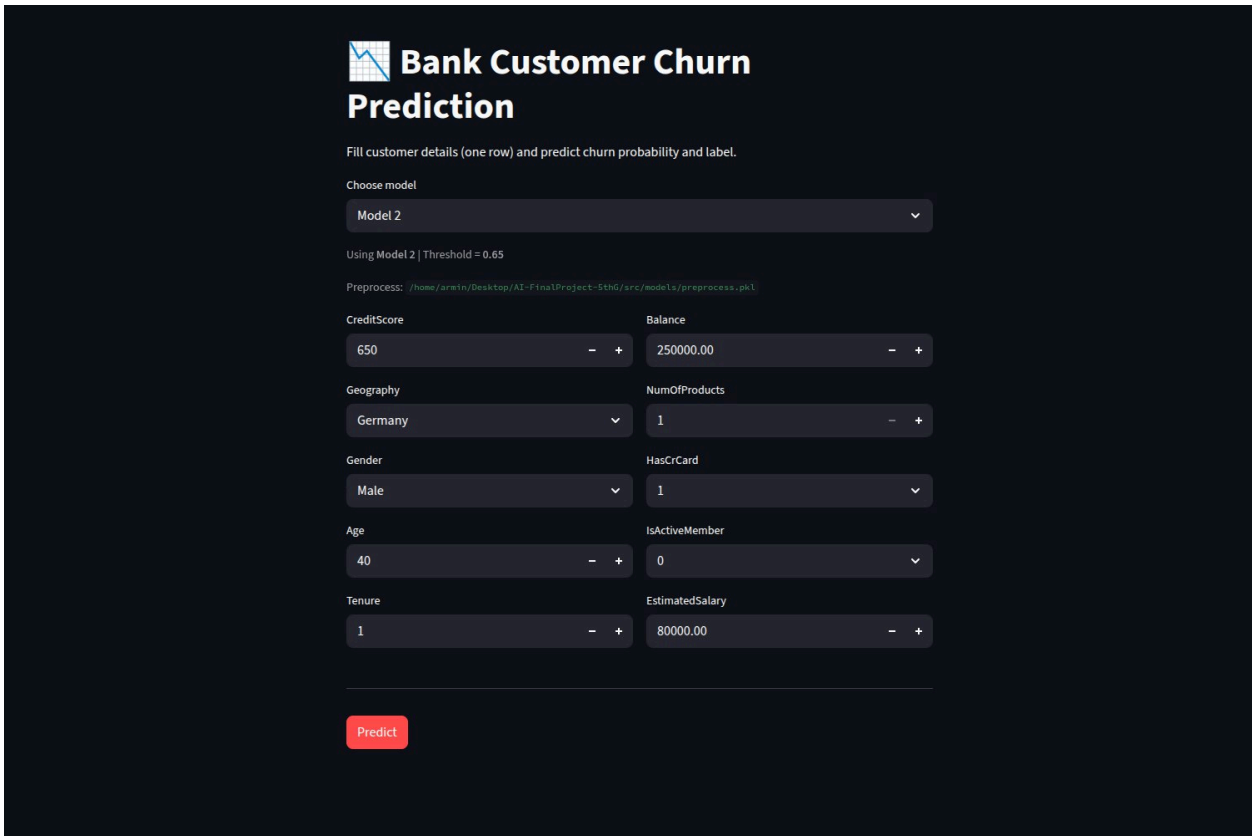
So despite different architecture and different imbalance strategies, they converged to **almost identical performance**.

This tell us that:

- The bottleneck is not the network size.
- The bottleneck is not the imbalance method.
- The bottleneck is feature separability.

When two very different modeling strategies converge to the same metric region, that usually means that: You are near the intrinsic limit of what the dataset allows.

DEMO SCREENSHOT



PERFORMANCE SUMMARY

The primary goal of this project was to build a churn prediction model that:

- Handles significant class imbalance (80% non-churn, 20% churn)
- Balances precision and recall
- Maximizes the F1-score
- Avoids excessive false positives or false negatives

The model was not optimized for recall alone, but for a balanced and stable performance across both classes.

Using the optimized threshold:

Overall Performance

- Accuracy: ~83%
- ROC-AUC: ~0.84
- Macro F1-score: ~0.75
- Weighted F1-score: ~0.83

Churn Class (Class 1)

- Precision: ~58–59%
- Recall: ~61–62%
- F1-score: ~0.60

An ROC-AUC of ~0.84 indicates that the model has good discriminative ability. It can effectively rank customers by churn risk.

The model achieves a well-balanced trade-off between:

- Detecting churn customers (recall)
- Avoiding excessive false alarms (precision)

This balance was intentionally optimized through threshold tuning to maximize F1-score.

From the confusion matrix:

- False Positive Rate ≈ 11%
- False Negative Rate ≈ 39%

While some churn customers are still missed, the model avoids aggressively over-predicting churn, maintaining stability.

The model:

- Does not collapse into majority-class bias
- Does not over-predict churn
- Maintains equilibrium between error types
- Generalizes well (no severe overfitting)
- Operates near its performance ceiling given the available features

It is important to note:

If recall were the only objective, we could increase it further by lowering the threshold — but this would significantly reduce precision and overall F1-score.

Since the objective was to maximize F1 and maintain balance, the current configuration represents an optimal operating point.

Given the dataset characteristics and feature space, the model has reached a justified and satisfactory performance level.